

MODULE 12

Convolutional Neural Networks CNN

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Convolutional Neural Networks CNN

What is a CNN?

CNNs are neural networks designed for image data.
They automatically learn spatial features.

Key components:

Convolutional Layer: apply filters to detect features

Pooling Layer: reduce spatial dimensions

Flatten: convert 2D to 1D

Fully Connected: classification/regression

Applications:

Image classification, object detection,
face recognition, medical imaging

CNN Architecture

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D(2, 2),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2, 2),
    Conv2D(128, (3,3), activation='relu'),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax') # 10 classes
])
```

Transfer Learning

```
from tensorflow.keras.applications import VGG16, ResNet50, MobileNetV2
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense
from tensorflow.keras.models import Model

# Load pretrained model without top
base_model = MobileNetV2(weights='imagenet', include_top=False,
input_shape=(224,224,3))
base_model.trainable = False # Freeze base layers
```

```
# Add custom head
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
output = Dense(num_classes, activation='softmax')(x)
model = Model(inputs=base_model.input, outputs=output)
```

Image Data Augmentation

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Data augmentation to prevent overfitting
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True
)

train_gen = train_datagen.flow_from_directory(
    'data/train', target_size=(224,224), batch_size=32, class_mode='categorical'
)
```

Object Detection

Object detection = locate + classify objects in images.

Popular frameworks:

YOLO (You Only Look Once): real-time detection

Faster R-CNN: accurate, slower

SSD (Single Shot Detector): balanced

```
from ultralytics import YOLO
model = YOLO('yolov8n.pt') # nano model
results = model('image.jpg')
for r in results:
    boxes = r.boxes
    print(boxes.xyxy, boxes.conf, boxes.cls)
```